



LeetCode 219: Contains Duplicate II

1. Problem Title & Link

Title: LeetCode 219: Contains Duplicate II

Link: <https://leetcode.com/problems/contains-duplicate-ii/>

2. Problem Statement (Short Summary)

Given:

- An integer array `nums`
- An integer `k`

Determine whether there exist two distinct indices `i` and `j` such that:

`nums[i] == nums[j]`

and

$|i - j| \leq k$

Return:

`true`

if such a pair exists, otherwise:

`false`

3. Examples (Input → Output)

Example 1

Input:

`nums = [1,2,3,1]`

`k = 3`

Output:

`true`

Explanation:

`nums[0] = 1`

`nums[3] = 1`

$|3 - 0| = 3$

Valid.

Example 2

Input:

`nums = [1,0,1,1]`



k = 1

Output:

true

Explanation:

nums[2] = 1

nums[3] = 1

distance = 1

Example 3

Input:

nums = [1,2,3,1,2,3]

k = 2

Output:

false

Explanation:

Duplicates exist

but distance > 2

4. Constraints

$1 \leq \text{nums.length} \leq 10^5$

$-10^9 \leq \text{nums}[i] \leq 10^9$

$0 \leq k \leq 10^5$

Important:

Need BOTH:

Same value

AND

Distance $\leq k$



5. Core Concept (Pattern / Topic)

HashMap

Secondary Topics:

- Sliding Window
- Last Seen Index

6. Thought Process (Step-by-Step Explanation)

Brute Force Thinking

For every element:

Check every other element.

Example:

[1,2,3,1]

Compare:

1 with 2

1 with 3

1 with 1

Found duplicate.

Calculate distance.

Complexity

Nested loops:

$O(n^2)$

Too slow.

Better Observation

We only need to know:

Where was this number seen last?

Example

Input:

[1,2,3,1]

k = 3

Process:



Index 0

1

Store:

1 → 0

Index 1

2

Store:

2 → 1

Index 2

3

Store:

3 → 2

Index 3

1

Already exists.

Previous index:

0

Distance:

$3 - 0 = 3$

Since:

$3 \leq k$

Return:

true

Key Insight

Store:

number → last index

Whenever duplicate appears:

currentIndex - previousIndex

Check if:

$\leq k$

7. Visual / Intuition Diagram (ASCII Diagram)

Input:



[1,2,3,1]

k = 3

Index

0 1 2 3

1 2 3 1

↑ ↑

Distance:

$3 - 0 = 3$

Valid.

Another Example

[1,2,3,1]

k = 2

Distance:

$3 - 0 = 3$

$3 > 2$

Not valid.

8. Pseudocode (Language Independent)

```
create hashmap
for each index i
    if number exists
        if i - previousIndex <= k
            return true
    update latest index
return false
```

9. Code Implementation

Python

```
class Solution:
```



```
def containsNearbyDuplicate(self, nums, k):  
  
    last_seen = {}  
  
    for i, num in enumerate(nums):  
  
        if num in last_seen:  
  
            if i - last_seen[num] <= k:  
                return True  
  
            last_seen[num] = i  
  
    return False
```

Java

```
import java.util.*;  
  
class Solution {  
  
    public boolean containsNearbyDuplicate(  
        int[] nums,  
        int k) {  
  
        HashMap<Integer,Integer> map =  
            new HashMap<>();  
  
        for(int i = 0; i < nums.length; i++) {  
  
            if(map.containsKey(nums[i])) {  
  
                if(i - map.get(nums[i]) <= k)  
                    return true;  
            }  
  
            map.put(nums[i], i);  
        }  
  
        return false;  
    }  
}
```

10. Time & Space Complexity

Time Complexity

$O(n)$

Single traversal.

**Space Complexity**

$O(n)$

HashMap may store all elements.

11. Common Mistakes / Edge Cases**Mistake 1**

Checking only duplicates.

Wrong:

Contains Duplicate

This problem additionally requires:

Distance $\leq k$

Mistake 2

Not updating latest index.

Example:

[1,0,1,1]

Need most recent occurrence.

Mistake 3

Using nested loops.

Leads to:

$O(n^2)$

Edge Cases

k = 0

Input:

[1,1]

k = 0

Output:

false

Because:

distance = 1

No Duplicates

Input:



[1,2,3,4]

Output:

false

12. Detailed Dry Run

Input:

nums = [1,0,1,1]

k = 1

Index	Value	HashMap	Action
0	1	{}	Store
1	0	{1:0}	Store
2	1	{1:0,0:1}	Distance=2
3	1	{1:2,0:1}	Distance=1 ✓

Return:

true

13. Common Use Cases (Real-Life / Interview)

Event Monitoring

Detect repeated events occurring close together.

Fraud Detection

Repeated transactions within a short interval.

Log Analysis

Duplicate messages within a limited window.

Interview Purpose

Introduces:

HashMap

Index Tracking

Distance Constraints

14. Common Traps (Important!)

Trap 1

Confusing with:



LC 217

Contains Duplicate

LC 217

Only asks:

Duplicate exists?

LC 219

Asks:

Duplicate exists

AND

Distance $\leq k$

Trap 2

Using a Set only.

Need index information.

Use:

HashMap

instead.

Trap 3

Forgetting absolute difference.

Since:

Current index > Previous index

we can simply use:

$i - \text{previousIndex}$

15. Builds To (Related LeetCode Problems)

LC 217 - Contains Duplicate



LC 219 - Contains Duplicate II



LC 220 - Contains Duplicate III



LC 3 - Longest Substring Without Repeating Characters



LC 904 - Fruit Into Baskets

These build Sliding Window + HashMap skills.

16. Alternate Approaches + Comparison

Approach	Time	Space	Interview Rating
Nested Loops	$O(n^2)$	$O(1)$	Poor
HashMap	$O(n)$	$O(n)$	Best
Sliding Window Set	$O(n)$	$O(k)$	Advanced

Approach 1: HashMap (Chosen)

Store last occurrence index.

Most common interview solution.

Approach 2: Sliding Window Set

Maintain only last k elements.

Useful advanced optimization.

17. Why This Solution Works (Short Intuition)

For every number, we only care about its most recent occurrence.

By storing:

value $\rightarrow$ last index

we can instantly calculate the distance between duplicate occurrences and determine whether it satisfies the condition.

18. Variations / Follow-Up Questions

Follow-Up 1

Return the duplicate pair indices.

Example:

[1,2,3,1]

$k = 3$

Output:

[0,3]



Follow-Up 2

Count all valid duplicate pairs.

Follow-Up 3

Find duplicates within a sliding window of size K.

Follow-Up 4

Solve:

LC 220

Contains Duplicate III

where both:

Value Difference

Index Difference

must satisfy constraints.

Interview Takeaway

The key pattern is:

HashMap + Last Seen Index

Whenever you hear:

Nearest occurrence

Previous occurrence

Distance between duplicates

think:

HashMap storing indices

This pattern is heavily used in Sliding Window, String Processing, and Log Analysis interview questions.